```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DS1302.h>
#include <ESP32Servo.h>
#include <HX711.h>
#include <EEPROM.h>
#include <Bounce2.h>

#include <WiFi.h>
#include "ThingSpeak.h"

const char* ssid       = "";
const char* password   = "";
unsigned long channelID = 3083537UL;
const char* writeAPIKey = "FFN26XGVSR0LGN5I";
const char* server     = "api.thingspeak.com";
bool lastDispenseSuccess = false;

WiFiClient client;

//=========================================================================
=======
// PIN DEFINITIONS & CONSTANTS
//=========================================================================
=======
#define TRIG_PIN        4
#define ECHO_PIN        2

#define RED_LED_PIN     27
#define YELLOW_LED_PIN  23
#define GREEN_LED_PIN   26

#define LID_SERVO_PIN   15
#define LID_OPEN_POS    90
#define LID_CLOSE_POS   0

#define BTN_UP_PIN      5
#define BTN_DOWN_PIN    32
#define BTN_OK_PIN      33
#define BTN_RETURN_PIN  25

#define DOUT_PIN        18
#define CLK_PIN         19
```

```
#define RTC_CLK_PIN      12
#define RTC_DATA_PIN     13
#define RTC_RST_PIN      17

#define MENU_TIMEOUT_MS  30000UL
#define LCD_MSG_DURATION 1500UL // Duration for messages like "Saved!" and "Done!"

// Debounce interval for Bounce2
#define NAV_DEBOUNCE_MS  15

// Ultrasonic calibration
#define US_SAMPLES       5
#define US_INTERVAL_MS   50UL

// Weight calibration
#define CAL_STEPS        20
#define SAMPLE_INTERVAL_MS (10000UL / CAL_STEPS)

// Dispense
const unsigned long DISPENSE_TIMEOUT_MS = 30000UL; // Your global timeout value in
milliseconds
#define DISPENSE_BURST_MS    2200 // How long the servo stays open in a burst
#define DISPENSE_WAIT_MS     500 // Wait between bursts for scale stability
#define DISPENSE_BUFFER_G    50.0f // New: The buffer in grams for dispensing tolerance
#define MIN_PROGRESS_PER_CHECK_G 10.0f // New: Require at least 1g of progress per
check.
#define BURSTS_PER_WEIGHT_CHECK 6 // New: Number of bursts before checking weight

// Servo Test
#define SERVO_TEST_DURATION_MS 10000 // Test for 10 seconds

//================================================================
=======
// GLOBAL OBJECTS & STATE
//================================================================
=======
LiquidCrystal_I2C lcd(0x27, 16, 2);
DS1302 rtc(RTC_RST_PIN, RTC_DATA_PIN, RTC_CLK_PIN);
Servo  lidServo;
HX711 scale;

// Bounce debouncers
```

```
Bounce btnUp     = Bounce();
Bounce btnDown   = Bounce();
Bounce btnOk     = Bounce();
Bounce btnReturn = Bounce();

// Meal definitions
enum { MEAL_BREAKFAST, MEAL_LUNCH, MEAL_DINNER, MEAL_COUNT };
const char* mealNames[MEAL_COUNT] = {"Breakfast","Lunch","Dinner"};
int   mealHour[MEAL_COUNT]   = {8,13,19};
int   mealMinute[MEAL_COUNT] = {0, 0, 0};
bool  mealDispensed[MEAL_COUNT] = {false,false,false};

// Persistent settings
float calib_factor = -3457.83f;
int   mealWeight   = 0;
int   calWeight    = 100;
int   usMin, usMax;

// UI/Menu
enum MenuState {
  STATE_DEFAULT,
  STATE_MAIN_MENU,
  STATE_SET_TIME,
  STATE_MEAL_SELECT_MENU,
  STATE_SET_MEAL,
  STATE_WEIGHT_MENU,
  STATE_SET_MEAL_WEIGHT,
  STATE_SET_CAL_WEIGHT,
  STATE_CALIBRATE_WEIGHT,
  STATE_TEST_WEIGHT,
  STATE_TEST_RAW,
  STATE_CAL_US,
  STATE_BUTTON_TEST,
  STATE_TEST_SERVO,
  STATE_TEST_DISPENSE
};
MenuState menuState      = STATE_DEFAULT;
int       menuIndex      = 0;
int       mealIndex      = 0;

String    lastButtonPressed;
unsigned long lastActivityMs = 0;
static uint32_t lastLedUpdateMs = 0;
```

```
// Editable scratch & cursors
int editHour     = 12, editMinute = 0;
int timeCursor   = 0;
int weightCursor = 0;

// Ultrasonic-cal state
enum USCalPhase {
  US_PROMPT_FULL,
  US_WAIT_FULL,
  US_SAMPLE_FULL,
  US_PROMPT_EMPTY,
  US_WAIT_EMPTY,
  US_SAMPLE_EMPTY
};
static USCalPhase usPhase = US_PROMPT_FULL;
static unsigned long usTs = 0;
static int usCount = 0;
static long usSum = 0;

// Weight-cal state
enum CalState { CAL_TARE, CAL_WAIT_STABILITY, CAL_PROMPT_PLACE,
CAL_TAKE_SAMPLE, CAL_COMPLETE };
static CalState calState = CAL_TARE;
static unsigned long calTs = 0;
static float lastCalVal = 0.0f;

// Test weight state machine
enum TestWeightState { TEST_IDLE, TEST_INIT, TEST_WAIT_FOR_STABILITY,
TEST_DISPLAY_READING };
static TestWeightState testWeightState = TEST_IDLE;
static unsigned long testWeightTs = 0;
static float lastTestWeightVal = 0.0f;

// Test raw state machine
enum TestRawState { TEST_RAW_IDLE, TEST_RAW_INIT, TEST_RAW_WAIT,
TEST_RAW_DISPLAY };
static TestRawState testRawState = TEST_RAW_IDLE;
static unsigned long testRawTs = 0;

// Servo Test state machine
enum ServoTestState { PROMPT, RUNNING, DONE, MESSAGE };
static ServoTestState servoTestState = PROMPT;
```

```
static unsigned long servoTestStartMs = 0;
static unsigned long lastToggleMs = 0;
static bool servoPosition = false; // false for closed, true for open

//=========================================================================
=======
// FORWARD DECLARATIONS
//=========================================================================
=======
void loadSettings();
void saveSettings();
void showSaveMessage();
void handleButtons();
void updateDisplay();
void handleTimeEditor(int &hour, int &minute);
void handleWeightEditor(int &weight, MenuState exitState);
void handleTestWeights();
void handleTestRaw();
void handleTestDispense();
void handleTestServo();
void handleCalUS();
void runCalibration();
void checkAndDispense();
void dispenseFood(float targetWeight);
long getRawDistance();
int  getFoodLevelPercent();
void updateLEDs(int pct);
void updateTimeEditorDisplay(const char* label, int hr, int mn, int cursor);
void updateWeightEditorDisplay(const char* label, int wt, int cursor);

//=========================================================================
=======
// EEPROM HELPERS
//=========================================================================
=======
void loadSettings() {
  EEPROM.begin(128);
  if (EEPROM.read(100) != 0x42) {
    EEPROM.write(100, 0x42);
    EEPROM.put(10, calib_factor);
    EEPROM.put(14, mealHour);
    EEPROM.put(14 + sizeof(mealHour), mealMinute);
    EEPROM.put(14 + sizeof(mealHour) + sizeof(mealMinute), mealWeight);
```

```
    EEPROM.put(14 + sizeof(mealHour) + sizeof(mealMinute) + sizeof(mealWeight), calWeight);
    EEPROM.put(50, 10);
    EEPROM.put(52, 60);
    EEPROM.commit();
  }
  EEPROM.get(10, calib_factor);
  EEPROM.get(14, mealHour);
  EEPROM.get(14 + sizeof(mealHour), mealMinute);
  EEPROM.get(14 + sizeof(mealHour) + sizeof(mealMinute) + sizeof(mealWeight), mealWeight);
  EEPROM.get(14 + sizeof(mealHour) + sizeof(mealMinute) + sizeof(mealWeight), calWeight);
  EEPROM.get(50, usMin);
  EEPROM.get(52, usMax);
}

void saveSettings() {
  EEPROM.put(10, calib_factor);
  EEPROM.put(14, mealHour);
  EEPROM.put(14 + sizeof(mealHour), mealMinute);
  EEPROM.put(14 + sizeof(mealHour) + sizeof(mealMinute), mealWeight);
  EEPROM.put(14 + sizeof(mealHour) + sizeof(mealMinute) + sizeof(mealWeight), calWeight);
  EEPROM.put(50, usMin);
  EEPROM.put(52, usMax);
  EEPROM.commit();
  showSaveMessage();
}

//=========================================================================
=======
// SETUP
//=========================================================================
=======
void setup() {
  Serial.begin(115200);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("WiFi connected");
  Serial.println(WiFi.status());
  ThingSpeak.begin(client);
```

```
  Wire.begin();
  loadSettings();

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(RED_LED_PIN, OUTPUT);
  pinMode(YELLOW_LED_PIN, OUTPUT);
  pinMode(GREEN_LED_PIN, OUTPUT);

  pinMode(BTN_UP_PIN,     INPUT_PULLUP);
  pinMode(BTN_DOWN_PIN,   INPUT_PULLUP);
  pinMode(BTN_OK_PIN,     INPUT_PULLUP);
  pinMode(BTN_RETURN_PIN, INPUT_PULLUP);

  btnUp.attach(BTN_UP_PIN);     btnUp.interval(NAV_DEBOUNCE_MS);
  btnDown.attach(BTN_DOWN_PIN); btnDown.interval(NAV_DEBOUNCE_MS);
  btnOk.attach(BTN_OK_PIN);     btnOk.interval(NAV_DEBOUNCE_MS);
  btnReturn.attach(BTN_RETURN_PIN); btnReturn.interval(NAV_DEBOUNCE_MS);

  lidServo.attach(LID_SERVO_PIN);
  lidServo.write(LID_CLOSE_POS);

  rtc.halt(false);
  rtc.writeProtect(false);

  scale.begin(DOUT_PIN, CLK_PIN);
  scale.set_scale(calib_factor);

  lcd.init();
  lcd.backlight();
  lcd.noAutoscroll();
  lcd.noBlink();
  lcd.clear();
  lcd.print("Feeder Ready");
  delay(LCD_MSG_DURATION);

  lcd.clear();
  lastActivityMs = millis();
}

void sendToThingSpeak(){
  // make sure we're still online
  if (WiFi.status() != WL_CONNECTED) {
```

```
    WiFi.reconnect();
    ThingSpeak.begin(client);
    Serial.println("Reconnecting to WiFi/ThingSpeak…");
    return;
  }

  // Example values from your feeder system
  int hour        = rtc.time().hr;
  int minute      = rtc.time().min;
  int foodLevel   = getFoodLevelPercent();  // ultrasonic sensor
  int currentMealW = ::mealWeight;
  int currentCalW  = ::calWeight;
  int usDistance  = getRawDistance();       // ultrasonic raw cm
  int servoStatus = (lidServo.read() > 45) ? 1 : 0;  // 1=open, 0=closed
  int dispenseOK  = lastDispenseSuccess ? 1 : 0;   // boolean flag

  // Assign values to ThingSpeak fields
  ThingSpeak.setField(1, hour);
  ThingSpeak.setField(2, minute);
  ThingSpeak.setField(3, foodLevel);
  ThingSpeak.setField(4, currentMealW);
  ThingSpeak.setField(5, currentCalW);
  ThingSpeak.setField(6, usDistance);
  ThingSpeak.setField(7, servoStatus);
  ThingSpeak.setField(8, dispenseOK);

  // Send to ThingSpeak
  int status = ThingSpeak.writeFields(channelID, writeAPIKey);

  if (status == 200) {
    Serial.println("✅ ThingSpeak write OK");
  } else {
    Serial.printf("⚠️ ThingSpeak write failed, HTTP %d\n", status);
  }
}

//=========================================================================
=======
// MAIN LOOP
//=========================================================================
=======
unsigned long lastSendMs = 0;
void loop() {
```

```
if (millis() - lastSendMs > 15000) {
  sendToThingSpeak();  // wrap the above code in a function
  lastSendMs = millis();
}

btnUp.update();
btnDown.update();
btnOk.update();
btnReturn.update();

// 2) Handle Navigation & Timeout
handleButtons();
if (menuState != STATE_DEFAULT &&
    millis() - lastActivityMs > MENU_TIMEOUT_MS) {
  menuState = STATE_DEFAULT;
}

// 3) Dispatch to Current State
switch (menuState) {
  case STATE_SET_TIME:
    handleTimeEditor(editHour, editMinute);
    break;
  case STATE_MEAL_SELECT_MENU:
    updateDisplay();
    break;
  case STATE_SET_MEAL:
    handleTimeEditor(mealHour[mealIndex], mealMinute[mealIndex]);
    break;
  case STATE_SET_MEAL_WEIGHT:
    handleWeightEditor(mealWeight, STATE_WEIGHT_MENU);
    break;

  case STATE_SET_CAL_WEIGHT:
    handleWeightEditor(calWeight, STATE_WEIGHT_MENU);
    break;

  case STATE_CALIBRATE_WEIGHT:
    runCalibration();
    break;

  case STATE_TEST_WEIGHT:
    handleTestWeights();
    break;
```

```
    case STATE_TEST_RAW:
      handleTestRaw();
      break;

    case STATE_TEST_DISPENSE:
      handleTestDispense();
      break;

    case STATE_TEST_SERVO:
      handleTestServo();
      break;

    case STATE_CAL_US:
      handleCalUS();
      break;

    default:
      updateDisplay();
      break;
  }

  // 4) Background Tasks
  checkAndDispense();
  if (millis() - lastLedUpdateMs > 10000UL) {
    updateLEDs(getFoodLevelPercent());
    lastLedUpdateMs = millis();
  }
}

//=========================================================================
=======
// EDIT TIME WITH SINGLE-STEP BUTTONS
//=========================================================================
=======
void handleTimeEditor(int &hour, int &minute) {
  static bool entered = false;
  if (!entered) {
    Serial.println(">>> Entering Time Editor");
    entered = true;
  }

  // Up/Down tweak
```

```
if (btnUp.fell()) {
  lastActivityMs = millis();
  if      (timeCursor == 0) hour   = (hour   + 10) % 24;
  else if (timeCursor == 1) hour   = (hour   +  1) % 24;
  else if (timeCursor == 2) minute = (minute + 10) % 60;
  else                      minute = (minute +  1) % 60;
}
if (btnDown.fell()) {
  lastActivityMs = millis();
  if      (timeCursor == 0) hour   = (hour   + 14) % 24;
  else if (timeCursor == 1) hour   = (hour   + 23) % 24;
  else if (timeCursor == 2) minute = (minute + 50) % 60;
  else                      minute = (minute + 59) % 60;
}

// Move cursor with OK
if (btnOk.fell()) {
  lastActivityMs = millis();
  timeCursor = (timeCursor + 1) % 4;
}

// RETURN → write back, show “Saved!”, exit to default
if (btnReturn.fell()) {
  lastActivityMs = millis();
  lcd.noBlink();

  if (menuState == STATE_SET_TIME) {
    Time tt = rtc.time();
    tt.hr  = hour;
    tt.min = minute;
    tt.sec = 0;
    rtc.time(tt);
    //Serial.printf("DEBUG: RTC.set → %02d:%02d:%02d\n",
     //            tt.hr, tt.min, tt.sec);
  }
  else {  // STATE_SET_MEAL
    mealHour[mealIndex]   = hour;
    mealMinute[mealIndex] = minute;
    //Serial.printf("DEBUG: Meal[%d] set → %02d:%02d\n",
      //           mealIndex, hour, minute);
  }

  showSaveMessage();      // flashes “Saved!” then clears
```

```
    menuState   = STATE_DEFAULT;
    entered     = false;    // reset for next edit
    return;                 // skip the redraw this cycle
  }

  // Redraw on every pass
  updateTimeEditorDisplay(
    (menuState == STATE_SET_TIME)
      ? "Set Time"
      : mealNames[mealIndex],
    hour, minute, timeCursor
  );
}


//=========================================================================
=======
// EDIT WEIGHT WITH SINGLE-STEP BUTTONS
//=========================================================================
=======
void handleWeightEditor(int &weight, MenuState exitState) {
  if (btnUp.fell()) {
    lastActivityMs = millis();
    if      (weightCursor == 0) weight = (weight + 100) % 1000;
    else if (weightCursor == 1) weight = (weight +  10) % 1000;
    else                        weight = (weight +   1) % 1000;
  }
  if (btnDown.fell()) {
    lastActivityMs = millis();
    if      (weightCursor == 0) weight = (weight + 900) % 1000;
    else if (weightCursor == 1) weight = (weight + 990) % 1000;
    else                        weight = (weight + 999) % 1000;
  }

  if (btnOk.fell()) {
    lastActivityMs = millis();
    weightCursor = (weightCursor + 1) % 3;
  }

  if (btnReturn.fell()) {
    lastActivityMs = millis();
    saveSettings();
    weightCursor = 0;
```

```
    menuState    = exitState;
  }

  const char* label = (&weight == &mealWeight)
    ? "Set Meal Weight"
    : "Set Cal Weight";

  updateWeightEditorDisplay(label, weight, weightCursor);
}

//=========================================================================
=======
// NAVIGATION HANDLER
//=========================================================================
=======
void handleButtons() {
  if (btnUp.fell()) {
    lastActivityMs    = millis();
    lastButtonPressed = "UP";

    if (menuState == STATE_DEFAULT) {
      menuState = STATE_MAIN_MENU;
      menuIndex = 0;
    }
    else if (menuState == STATE_MAIN_MENU) {
      menuIndex = (menuIndex + 2) % 3;
    }
    else if (menuState == STATE_MEAL_SELECT_MENU) {
      mealIndex = (mealIndex + MEAL_COUNT - 1) % MEAL_COUNT;
    }
    else if (menuState == STATE_WEIGHT_MENU) {
      weightCursor = (weightCursor + 7) % 8;
    }
  }

  if (btnDown.fell()) {
    lastActivityMs    = millis();
    lastButtonPressed = "DOWN";

    if (menuState == STATE_DEFAULT) {
      menuState = STATE_MAIN_MENU;
      menuIndex = 0;
    }
```

```
  else if (menuState == STATE_MAIN_MENU) {
    menuIndex = (menuIndex + 1) % 3;
  }
  else if (menuState == STATE_MEAL_SELECT_MENU) {
    mealIndex = (mealIndex + 1) % MEAL_COUNT;
  }
  else if (menuState == STATE_WEIGHT_MENU) {
    weightCursor = (weightCursor + 1) % 8;
  }
}

if (btnOk.fell()) {
  lastActivityMs    = millis();
  lastButtonPressed = "OK";

  if (menuState == STATE_DEFAULT) {
    menuState = STATE_MAIN_MENU;
    menuIndex = 0;
  }
  else if (menuState == STATE_MAIN_MENU) {
    if (menuIndex == 0) {
      Time now = rtc.time();
      editHour   = now.hr;
      editMinute = now.min;
      timeCursor = 0;
      menuState  = STATE_SET_TIME;
    }
    else if (menuIndex == 1) {
      mealIndex = 0;
      menuState = STATE_MEAL_SELECT_MENU;
    }
    else {
      weightCursor = 0;
      menuState    = STATE_WEIGHT_MENU;
    }
  }
  else if (menuState == STATE_MEAL_SELECT_MENU) {
    editHour   = mealHour[mealIndex];
    editMinute = mealMinute[mealIndex];
    timeCursor = 0;
    menuState  = STATE_SET_MEAL;
  }
  else if (menuState == STATE_WEIGHT_MENU) {
```

```
    switch (weightCursor) {
      case 0:  menuState = STATE_SET_MEAL_WEIGHT;    break;
      case 1:  menuState = STATE_SET_CAL_WEIGHT;     break;
      case 2:  testWeightState = TEST_INIT;
               menuState       = STATE_TEST_WEIGHT;  break;
      case 3:  testRawState    = TEST_RAW_INIT;
               menuState       = STATE_TEST_RAW;     break;
      case 4:  menuState       = STATE_CAL_US;       break;
      case 5:  menuState       = STATE_CALIBRATE_WEIGHT; break;
      case 6:  servoTestState  = PROMPT;
               menuState       = STATE_TEST_SERVO;   break;
      case 7:  menuState       = STATE_TEST_DISPENSE;  break;
    }
    weightCursor = 0;
  }
}

// Only handle RETURN if NOT currently in a time-edit state
if (btnReturn.fell()
    && menuState != STATE_SET_TIME
    && menuState != STATE_SET_MEAL) {

  lastActivityMs    = millis();
  lastButtonPressed = "RETURN";

  switch (menuState) {
    case STATE_MEAL_SELECT_MENU:
    case STATE_SET_MEAL_WEIGHT:
    case STATE_SET_CAL_WEIGHT:
    case STATE_CALIBRATE_WEIGHT:
    case STATE_TEST_WEIGHT:
    case STATE_TEST_RAW:
    case STATE_CAL_US:
    case STATE_TEST_SERVO:
    case STATE_TEST_DISPENSE:
      menuState = STATE_WEIGHT_MENU;
      break;

    case STATE_WEIGHT_MENU:
    case STATE_MAIN_MENU:
    case STATE_DEFAULT:
      menuState = STATE_DEFAULT;
      break;
```

```
      default:
        break;
    }
  }
}


//=========================================================================
=======
// ULTRASONIC CALIBRATION
//=========================================================================
=======
void handleCalUS() {
  unsigned long now = millis();
  switch (usPhase) {
    case US_PROMPT_FULL:
      lcd.clear();
      lcd.setCursor(0,0);
      lcd.print("Move to FULL");
      lcd.setCursor(0,1);
      lcd.print("[OK] when ready");
      usPhase = US_WAIT_FULL;
      break;

    case US_WAIT_FULL:
      if (btnReturn.fell()) {
        menuState = STATE_WEIGHT_MENU;
        usPhase   = US_PROMPT_FULL;
      }
      else if (btnOk.fell()) {
        usTs    = now;
        usCount = 0;
        usSum   = 0;
        lcd.clear();
        lcd.setCursor(0,0);
        lcd.print("Calibrating...");
        lcd.setCursor(0,1);
        lcd.print('[');
        for (int i=0; i<US_SAMPLES; ++i) lcd.print(' ');
        lcd.print(']');
        usPhase = US_SAMPLE_FULL;
      }
```

```
  break;

case US_SAMPLE_FULL:
  if (now - usTs >= US_INTERVAL_MS) {
    usTs = now;
    usSum += getRawDistance();
    lcd.setCursor(++usCount,1);
    lcd.print('#');
    if (usCount >= US_SAMPLES) {
      usMin   = usSum / US_SAMPLES;
      usPhase = US_PROMPT_EMPTY;
    }
  }
  break;

case US_PROMPT_EMPTY:
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("Move to EMPTY");
  lcd.setCursor(0,1);
  lcd.print("[OK] when ready");
  usPhase = US_WAIT_EMPTY;
  break;

case US_WAIT_EMPTY:
  if (btnReturn.fell()) {
    menuState = STATE_WEIGHT_MENU;
    usPhase   = US_PROMPT_FULL;
  }
  else if (btnOk.fell()) {
    usTs    = now;
    usCount = 0;
    usSum   = 0;
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("Calibrating...");
    lcd.setCursor(0,1);
    lcd.print('[');
    for (int i=0; i<US_SAMPLES; ++i) lcd.print(' ');
    lcd.print(']');
    usPhase = US_SAMPLE_EMPTY;
  }
  break;
```

```
    case US_SAMPLE_EMPTY:
      if (now - usTs >= US_INTERVAL_MS) {
        usTs = now;
        usSum += getRawDistance();
        lcd.setCursor(++usCount,1);
        lcd.print('#');
        if (usCount >= US_SAMPLES) {
          usMax = usSum / US_SAMPLES;
          saveSettings();
          menuState = STATE_WEIGHT_MENU;
          usPhase   = US_PROMPT_FULL;
        }
      }
      break;
  }
}

//=========================================================================
=======
// WEIGHT CALIBRATION
//=========================================================================
=======
void runCalibration() {
  unsigned long now = millis();
  const unsigned long CAL_DISPLAY_INTERVAL = 1000;
  const unsigned long STABILITY_WAIT_MS = 500;
  const int VALID_SAMPLE_COUNT = 5;
  static int validSamplesCount = 0;
  static long validSamplesSum = 0;

  switch (calState) {
    case CAL_TARE:
      lcd.clear(); lcd.setCursor(0,0);
      lcd.print("Taring scale...");
      scale.power_down();
      scale.power_up();
      calTs = now;
      calState = CAL_WAIT_STABILITY;
      break;

    case CAL_WAIT_STABILITY:
      if (now - calTs >= STABILITY_WAIT_MS) {
```

```
    scale.tare();
    calState = CAL_PROMPT_PLACE;
  }
  break;

case CAL_PROMPT_PLACE:
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.printf("Place %dg", calWeight);
  lcd.setCursor(0,1);
  lcd.print("on the scale");

  if (btnOk.fell()) {
    calState = CAL_TAKE_SAMPLE;
    calTs = now;
    validSamplesCount = 0;
    validSamplesSum = 0;
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("Sampling...");
  }
  if (btnReturn.fell()) {
    menuState = STATE_WEIGHT_MENU;
    calState = CAL_TARE;
  }
  break;

case CAL_TAKE_SAMPLE:
  if (now - calTs >= CAL_DISPLAY_INTERVAL) {
    calTs = now;
    long currentRawValue = scale.get_value(20);

    if (abs(currentRawValue) > 10000000) {
        Serial.println("Warning: Corrupted reading detected. Skipping this sample.");
        lcd.clear();
        lcd.setCursor(0,0);
        lcd.print("Bad reading. Retrying...");
        lcd.setCursor(0,1);
        lcd.print("Don't touch scale.");
        calTs = now;
        return;
    }
```

```
      validSamplesSum += currentRawValue;
      validSamplesCount++;
      lastCalVal = currentRawValue;

      Serial.print("Calibration Raw Value: ");
      Serial.println(lastCalVal);

      lcd.setCursor(0,0);
      lcd.print("Sampling...");
      lcd.setCursor(0,1);
      char buf[20];
      sprintf(buf, "%ld", currentRawValue);
      lcd.print(buf);

      lcd.setCursor(14,0);
      lcd.print("[");
      lcd.setCursor(15,0);
      lcd.print(validSamplesCount);
      lcd.print("]");
    }

    if (validSamplesCount >= VALID_SAMPLE_COUNT) {
       float averageRawValue = (float)validSamplesSum / validSamplesCount;
       if (calWeight > 0) {
         calib_factor = averageRawValue / calWeight;
         scale.set_scale(calib_factor);
         Serial.print("New Scale Factor: ");
         Serial.println(calib_factor, 2);
         if (calib_factor > 0) {
           Serial.println("Note: A positive scale factor means your load cell is wired 'backwards'.
The code will now correct for this, but if you want positive readings without a negative scale
factor, flip the load cell's signal wires.");
         }
       }
       saveSettings();
       calState = CAL_COMPLETE;
       calTs = now;
    }

    if (btnReturn.fell()) {
      menuState = STATE_WEIGHT_MENU;
      calState = CAL_TARE;
    }
```

```
      break;

    case CAL_COMPLETE:
      lcd.clear();
      lcd.setCursor(0,0);
      lcd.print("Complete!");
      if (now - calTs >= LCD_MSG_DURATION) {
        menuState = STATE_WEIGHT_MENU;
        calState = CAL_TARE;
      }
      break;
  }
}

//=========================================================================
=======
// TEST WEIGHTS
//=========================================================================
=======
void handleTestWeights() {
  const unsigned long POWER_UP_DELAY_MS = 500;
  const unsigned long TEST_INTERVAL_MS  = 500;

  unsigned long now = millis();

  switch(testWeightState) {
    case TEST_INIT:
      scale.power_down();
      scale.power_up();
      testWeightTs = now;
      lcd.clear();
      lcd.setCursor(0,0);
      lcd.print("Warming up...");
      testWeightState = TEST_WAIT_FOR_STABILITY;
      break;

    case TEST_WAIT_FOR_STABILITY:
      if (now - testWeightTs >= POWER_UP_DELAY_MS && scale.is_ready()) {
        testWeightState = TEST_DISPLAY_READING;
        testWeightTs = now;
        lcd.clear();
      }
      break;
```

```
    case TEST_DISPLAY_READING:
      if (now - testWeightTs >= TEST_INTERVAL_MS) {
        testWeightTs = now;
        float v = scale.get_units(20);

        if (fabs(v) > 1000000.0f) {
          Serial.println("Warning: Corrupted reading detected. Skipping this sample.");
          return;
        }

        lastTestWeightVal = v;

        Serial.print("Test Weight: ");
        Serial.print(lastTestWeightVal, 2);
        Serial.println(" g");

        lcd.setCursor(0,0);
        lcd.print(" Test Weights ");
        lcd.setCursor(0,1);
        if (isnan(lastTestWeightVal)) {
          lcd.print("---.-- g");
        }
        else {
          char buf[20];
          dtostrf(lastTestWeightVal, 6, 2, buf);
          lcd.print(buf);
          lcd.print(" g");
        }
      }
      break;

    default:
      break;
  }

  if (btnReturn.fell()) {
    testWeightState = TEST_IDLE;
    menuState = STATE_WEIGHT_MENU;
    lcd.clear();
  }
}
```

```
//=========================================================================
=======
// TEST RAW
//=========================================================================
=======
void handleTestRaw() {
  const unsigned long TEST_RAW_INTERVAL_MS  = 500;
  unsigned long now = millis();

  switch(testRawState) {
    case TEST_RAW_INIT:
      scale.power_down();
      scale.power_up();
      testRawTs = now;
      testRawState = TEST_RAW_WAIT;
      lcd.clear();
      lcd.setCursor(0,0);
      lcd.print("Warming up...");
      break;

    case TEST_RAW_WAIT:
      if (now - testRawTs > 500UL) {
        testRawState = TEST_RAW_DISPLAY;
        testRawTs = now;
        lcd.clear();
      }
      break;

    case TEST_RAW_DISPLAY:
      if (now - testRawTs >= TEST_RAW_INTERVAL_MS) {
        testRawTs = now;
        long rawValue = scale.read();
        Serial.printf("Raw Value: %ld\n", rawValue);

        lcd.setCursor(0,0);
        lcd.print("Raw Value:");
        lcd.setCursor(0,1);
        char buf[20];
        sprintf(buf, "%ld", rawValue);
        lcd.print(buf);
      }
      break;
```

```
    default:
      break;
  }

  if (btnReturn.fell()) {
    testRawState = TEST_RAW_IDLE;
    menuState = STATE_WEIGHT_MENU;
    lcd.clear();
  }
}

//=========================================================================
=======
// TEST DISPENSE (NOW USES BLOCKING LOGIC)
//=========================================================================
=======
void handleTestDispense() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Test Dispense");
  lcd.setCursor(0, 1);
  lcd.print("OK to start");

  // Check for OK button press to start the blocking sequence
  while (true) {
    btnOk.update();
    btnReturn.update();

    if (btnOk.fell()) {
      if (mealWeight == 0) {
        lcd.clear();
        lcd.setCursor(0,0);
        lcd.print("Meal weight is 0!");
        delay(LCD_MSG_DURATION);
      } else {
        dispenseFood(mealWeight);
      }
      break;
    }

    if (btnReturn.fell()) {
      break;
    }
```

```
  }
  menuState = STATE_WEIGHT_MENU;
  lcd.clear();
}

//=========================================================================
=======
// TEST SERVO
//=========================================================================
=======
void handleTestServo() {
  static bool prompted = false;

  // Wait for OK to start
  if (!prompted) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Test Servo");
    lcd.setCursor(0, 1);
    lcd.print("OK to start");
    prompted = true;
    return;
  }

  if (btnOk.fell()) {
    prompted = false; // Reset for next time

    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("Testing...");

    // Blocking loop for the servo test
    unsigned long startTime = millis();
    while (millis() - startTime < SERVO_TEST_DURATION_MS) {
      // Open the servo
      lidServo.write(LID_OPEN_POS);
      Serial.println("Servo OPEN (blocking)");
      delay(1000); // Wait for 1 second

      // Close the servo
      lidServo.write(LID_CLOSE_POS);
      Serial.println("Servo CLOSED (blocking)");
      delay(1000); // Wait for 1 second
```

```
      // Check for return button press to exit early
      btnReturn.update();
      if (btnReturn.fell()) {
        lidServo.write(LID_CLOSE_POS);
        menuState = STATE_WEIGHT_MENU;
        lcd.clear();
        return;
      }
    }

    lcd.clear();
    lcd.print("Test Complete!");
    delay(LCD_MSG_DURATION);

    menuState = STATE_WEIGHT_MENU;
  }

  if (btnReturn.fell()) {
    prompted = false;
    menuState = STATE_WEIGHT_MENU;
    lcd.clear();
  }
}

//=========================================================================
=======
// DRAW THE TIME EDITOR PROMPT + CURSOR
//=========================================================================
=======
void updateTimeEditorDisplay(const char* label, int hr, int mn, int cursor) {
  lcd.noBlink();
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print(label);

  lcd.setCursor(0,1);
  if (hr < 10) lcd.print('0');
  lcd.print(hr);
  lcd.print(':');
  if (mn < 10) lcd.print('0');
  lcd.print(mn);
```

```
  // Skip colon in cursor positioning
  int pos = (cursor < 2) ? cursor : (cursor + 1);
  lcd.setCursor(pos,1);
  lcd.blink();
}

//=========================================================================
=======
// DRAW THE WEIGHT EDITOR PROMPT + CURSOR
//=========================================================================
=======
void updateWeightEditorDisplay(const char* label, int wt, int cursor) {
  lcd.noBlink();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print(label);

  lcd.setCursor(0, 1);
  char buf[6];
  sprintf(buf, "%03dg", wt);
  lcd.print(buf);

  lcd.setCursor(cursor, 1);
  lcd.blink();
}

//=========================================================================
=======
// DISPLAY & HELPERS
//=========================================================================
=======
void updateDisplay() {
  lcd.noBlink();
  lcd.clear();
  Time t = rtc.time();
  //Serial.printf("DEBUG: RTC.read → %02d:%02d:%02d\n", t.hr, t.min, t.sec);

  switch (menuState) {
    case STATE_DEFAULT:
      lcd.setCursor(0,0);
      lcd.printf("Time %02d:%02d", t.hr, t.min);
      lcd.setCursor(0,1);
      lcd.printf("Food %d%%", getFoodLevelPercent());
```

```
  break;

case STATE_MAIN_MENU: {
  static const char* items[] = {
    "Device Time",
    "Meal Time",
    "Weights & Calib"
  };
  lcd.setCursor(0,0);
  lcd.print("> ");
  lcd.print(items[menuIndex]);
  lcd.setCursor(0,1);
  lcd.print("OK=Select  ");
  break;
}

case STATE_MEAL_SELECT_MENU:
  lcd.setCursor(0,0);
  lcd.print("> ");
  lcd.print(mealNames[mealIndex]);
  lcd.setCursor(0,1);
  lcd.print("OK=Edit Time");
  break;

case STATE_WEIGHT_MENU: {
  static const char* items[] = {
    "Set Meal Weight",
    "Set Cal Weight",
    "Test Weights",
    "Test Raw",
    "Cal Ultrasonic",
    "Cal Weight",
    "Test Servo",
    "Test Dispense" // New menu item
  };
  lcd.setCursor(0,0);
  lcd.print("> ");
  lcd.print(items[weightCursor]);
  lcd.setCursor(0,1);
  lcd.print("OK=Select");
  break;
}
```

```
    case STATE_BUTTON_TEST:
      lcd.setCursor(0,0);
      lcd.print("Button Pressed:");
      lcd.setCursor(0,1);
      lcd.print(lastButtonPressed);
      break;

    default:
      break;
  }
}

long getRawDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long dur = pulseIn(ECHO_PIN, HIGH, 20000);
  return dur * 0.034f / 2;
}

int getFoodLevelPercent() {
  long d = getRawDistance();
  if (d < 2 || d > 100) return 0;
  d = constrain(d, usMin, usMax);
  return map(d, usMax, usMin, 0, 100);
}

void updateLEDs(int pct) {
  digitalWrite(GREEN_LED_PIN,  pct > 70 ? HIGH : LOW);
  digitalWrite(YELLOW_LED_PIN, pct > 30 && pct <= 70 ? HIGH : LOW);
  digitalWrite(RED_LED_PIN,    pct <= 30 ? HIGH : LOW);
}

void showSaveMessage() {
  lcd.noBlink();
  lcd.clear();
  lcd.print("Saved!");
  unsigned long t0 = millis();
  while (millis() - t0 < LCD_MSG_DURATION) {
    btnUp.update(); btnDown.update();
    btnOk.update(); btnReturn.update();
```

```
  }
  lcd.clear();
}

void shutdown() {
  lidServo.write(LID_CLOSE_POS);
  saveSettings();
  lcd.clear();
  lcd.print("Shutting down...");
  unsigned long startWait = millis();
  while (millis() - startWait < LCD_MSG_DURATION) {
    btnUp.update();
    btnDown.update();
    btnOk.update();
    btnReturn.update();
  }
}

//=========================================================================
=======
// BLOCKING DISPENSE FUNCTION
//=========================================================================
=======
void dispenseFood(float targetWeight) {
  float initialWeight = scale.get_units(5);
  float lastMeasuredWeight = initialWeight;
  unsigned long startTime = millis();
  unsigned long lastProgressTime = millis();
  int burstCounter = 0;

  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("Dispensing...");
  Serial.println("**** DISPENSE STARTED ****");

  float dispensedWeight;

  while ( (scale.get_units(5) - initialWeight) < targetWeight && (millis() - startTime) <
DISPENSE_TIMEOUT_MS) {
    // Open the servo for a short burst
    lidServo.write(LID_OPEN_POS);
    delay(DISPENSE_BURST_MS);
```

```
  // Close the servo and wait for the scale to stabilize
  lidServo.write(LID_CLOSE_POS);
  delay(DISPENSE_WAIT_MS);

  burstCounter++;

  // Only check for progress after a set number of bursts
  if (burstCounter >= BURSTS_PER_WEIGHT_CHECK) {
    float currentMeasuredWeight = scale.get_units(5);

    if ((currentMeasuredWeight - lastMeasuredWeight) < MIN_PROGRESS_PER_CHECK_G) {
        lcd.clear();
        lcd.setCursor(0,0);
        lcd.print("Dispense Error!");
        lastDispenseSuccess = false;
        Serial.println("Dispensing failed. No weight added in this burst.");
        delay(LCD_MSG_DURATION);
        break; // Exit the loop and stop trying to dispense
    }

    // If we made progress, reset the counter and update the progress time
    burstCounter = 0;
    lastMeasuredWeight = currentMeasuredWeight;
    lastProgressTime = millis();
  }

  // Update LCD and serial with current progress
  dispensedWeight = scale.get_units(5) - initialWeight;
  lcd.setCursor(0,1);
  lcd.printf("W:%.1f T:%.1f g", dispensedWeight, targetWeight);
  Serial.printf("Current Weight: %.1f g, Target: %.1f g\n", dispensedWeight, targetWeight);
}

// Final check and messages
lidServo.write(LID_CLOSE_POS); // Ensure servo is closed at the end

dispensedWeight = scale.get_units(5) - initialWeight;

if ( (millis() - lastProgressTime) >= DISPENSE_TIMEOUT_MS ) {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("Timeout!");
  lastDispenseSuccess = false;
```

```
    Serial.println("**** DISPENSE TIMEOUT ERROR ****");
  } else if (abs(dispensedWeight - targetWeight) <= DISPENSE_BUFFER_G) {
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("Done! (In buffer)");
    lastDispenseSuccess = true;
    Serial.printf("**** DISPENSE COMPLETE (IN BUFFER) - Final Weight: %.1f g ****\n",
dispensedWeight);
  } else {
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("Done! (Overshot)");
    lastDispenseSuccess = true;
    Serial.printf("**** DISPENSE COMPLETE (OVERSHOT) - Final Weight: %.1f g ****\n",
dispensedWeight);
  }

  delay(LCD_MSG_DURATION); // Show message for a moment
}

//=========================================================================
=======
// DISPENSE HELPER (STATE MACHINE)
//=========================================================================
=======
void checkAndDispense() {
  Time t = rtc.time();
  for (int i = 0; i < MEAL_COUNT; ++i) {
    // If it's time for a meal, it hasn't been dispensed yet, and a meal weight is set
    if (t.hr == mealHour[i] && t.min == mealMinute[i] && !mealDispensed[i] && mealWeight > 0) {
      dispenseFood(mealWeight);
      mealDispensed[i] = true; // Mark as dispensed
      break;
    }
  }

  // Reset dispense flags at midnight
  if (t.hr == 0 && t.min == 0) {
    for (int i = 0; i < MEAL_COUNT; ++i) {
      mealDispensed[i] = false;
    }
  }
}
```